Overall FMM Sequence for Eshelby Particles Used in ParaDiS

For Eshelby particles in ParaDiS, the FMM sequence is basically a way to compute the **far-field stress from many inclusions efficiently**. Instead of evaluating every particle–target interaction directly, ParaDiS groups particles into cells, summarizes each cell by multipole moments, translates those moments up the tree, converts distant multipoles into local Taylor expansions, translates those local expansions down the tree, and finally evaluates stress at the target point.

The overall sequence in the documentation is

$$\boxed{\text{MakeEta} \rightarrow \text{TranslateEta} \rightarrow \text{DerofR} + \text{MakeTaylor} \rightarrow \text{TranslateTaylor} \rightarrow \text{EvalTaylor}}$$

and the direct `EshelbyForce.cc` path is separate from this far-field FMM chain.

1. Start from individual Eshelby particles

Each spherical Eshelby inclusion has:

- center x_s,
- radius a,
- volumetric strain

$$E_v = \epsilon_{xx} + \epsilon_{yy} + \epsilon_{zz}.$$

The FMM uses the source strength

$$\boxed{q = E_v a^3}$$

and the far-field kernel

$$\boxed{K_{ij}(r) = \frac{\delta_{ij}}{r^3} - \frac{3r_i r_j}{r^5}.}$$

So a single inclusion contributes, at monopole level,

$$\sigma_{ij} = A(\mu, \nu) E_v a^3 K_{ij}(r),$$

where

$$A(\mu, \nu) = \frac{\mu(1+\nu)}{1-2\nu}.$$

But ParaDiS does not want to evaluate this separately for every distant particle. That is where the FMM tree comes in.

2. P2M: particle to multipole

The first step is

`EshelbyMakeEta`

which converts each inclusion into multipole coefficients about the center of its leaf cell.

If the leaf-cell center is c_s, then

$$d = c_s - x_s.$$

The particle generates moments

$$\boxed{M_\alpha = E_v a^3 d^\alpha}$$

with

$$d^\alpha = d_x^{\alpha_x} d_y^{\alpha_y} d_z^{\alpha_z}.$$

For example,

$$M_0 = q,$$

$$(M_x, M_y, M_z) = q(d_x, d_y, d_z),$$

and

$$(M_{xx}, M_{xy}, \ldots) = q(d_x^2, d_x d_y, \ldots).$$

For several particles in the same leaf cell, ParaDiS sums these:

$$\boxed{M_\alpha^{cell} = \sum_s E_{v,s} a_s^3 (c_s - x_s)^\alpha.}$$

So many physical particles become one compact multipole description.

Conceptually:

```
particle 1 -+
particle 2 -+--> multipole expansion of leaf cell
particle 3 -+
```

This is the **P2M step**.

3. M2M: move child multipoles to parent cells

Next comes

`EshelbyTranslateEta`

which takes the multipole expansion from a child cell and translates it to the parent cell center. This is the upward pass of the FMM tree.

Conceptually:

```
leaf cells
   v
parent cells
   v
larger parent cells
   v
root
```

The mathematical operation is a multidimensional binomial translation:

$$\boxed{M'_\alpha = \sum_{\beta \le \alpha} \binom{\alpha}{\beta} M_\beta h^{\alpha-\beta}}$$

for an appropriate center shift h. For example,

$$M'_x = M_x + M_0 h_x,$$

and

$$M'_{xx} = M_{xx} + 2M_x h_x + M_0 h_x^2.$$

So multiple child-cell expansions can be expressed about the same parent center and then summed. The implementation uses a hard-coded multinomial table through order 6.

A useful picture is:

```
child A multipoles -+
child B multipoles -+- TranslateEta -> parent multipoles
child C multipoles -+
```

This is **M2M**.

4. Determine which source cells are sufficiently far away

Once the tree contains multipole descriptions at different levels, ParaDiS decides which source cells are far enough from a target cell to use the FMM approximation. For far-separated cells, the detailed individual particles are no longer needed. Instead of

$$\text{particle} \to \text{target},$$

ParaDiS evaluates

$$\text{source-cell multipole} \to \text{target-cell local expansion}.$$

That is the central speedup.

5. Compute derivatives of the interaction kernel

For a source-cell center c_s and target-cell center c_t, define

$$\boxed{R = c_t - c_s.}$$

Then ParaDiS calls

$$\boxed{\texttt{EshelbyDerofR}}$$

to generate the derivatives

$$\boxed{D_{i_1 \ldots i_n}(R) = -\partial_{i_1} \cdots \partial_{i_n} \frac{1}{R}.}$$

The lowest rank is

$$D_{ij} = \frac{\delta_{ij}}{R^3} - \frac{3R_i R_j}{R^5},$$

which is exactly the Eshelby FMM kernel. Higher ranks,

$$D_{ijk}, D_{ijkl}, \ldots,$$

are required to couple higher multipole orders to higher Taylor orders. So:

$$\boxed{\texttt{DerofR} = \text{geometry between source and target cells.}}$$

6. `EshelbyBuildMatrix`: organize the derivative tensor

The derivative list is packed into a one-dimensional symmetric-tensor representation. `EshelbyBuildMatrix` rearranges selected derivative components into matrices suitable for contractions with multipole vectors. Mathematically, it constructs

$$\boxed{B_{\alpha\beta} = D_{\alpha+\beta}.}$$

Here:

- α corresponds to a target Taylor derivative,
- β corresponds to a source multipole moment.

So this function is mostly an indexing/packing helper.

7. M2L: multipole to local Taylor expansion

Then comes one of the most important routines:

`EshelbyMakeTaylor`

This converts the source-cell multipole expansion into a local Taylor expansion about the target-cell center. Mathematically:

$$\boxed{L_{ij,\alpha} = A(\mu,\nu)\sum_{\beta} D_{ij,\alpha+\beta}(R)\frac{M_\beta}{\beta!}.}$$

This is the **M2L step**.

Interpretation:

- M_β: what is inside the source cell,
- $D_{ij,\alpha+\beta}(R)$: how the field propagates from source-cell center to target-cell center,
- $L_{ij,\alpha}$: local representation of that distant source field near the target cell.

Conceptually:

```
source-cell multipoles
          +
source-target separation
          v
     EshelbyMakeTaylor
          v
target-cell local Taylor expansion
```

This is the key conversion from the **source representation** to the **target representation**.

8. Accumulate contributions from many distant cells

A target cell may interact with many well-separated source cells. Each source contributes a Taylor expansion:

$$L^{(1)}, L^{(2)}, L^{(3)}, \ldots$$

These can simply be added:

$$\boxed{L^{target} = \sum_{s} L^{(s)}.}$$

So one target cell eventually contains a local expansion representing the combined stress field from many distant source groups.

9. L2L: move the local expansion downward

The target-cell Taylor expansion may initially exist at a relatively coarse parent cell. But ultimately stress is required in a much smaller child cell. So ParaDiS calls

`EshelbyTranslateTaylor`

to translate the Taylor expansion from parent center c_p to child center c_c. Mathematically, the intended translation is

$$L_{ij,\alpha}^{child} = \sum_{\beta} L_{ij,\alpha+\beta}^{parent} \frac{(c_c - c_p)^{\beta}}{\beta!}.$$

This is the **L2L step**.

Conceptually:

```
parent local expansion
        v
TranslateTaylor
        v
child local expansion
        v
TranslateTaylor
        v
leaf local expansion
```

So the upward pass moves **multipoles upward**, while the downward pass moves **Taylor expansions downward**.

10. L2P: evaluate the Taylor expansion at the actual point

Once the expansion has reached the target leaf cell, ParaDiS finally evaluates it at the physical target point x. That is done by

`EshelbyEvalTaylor`.

If the target-cell center is c_t,

$$y = x - c_t.$$

The stress is

$$\sigma_{ij}(x) = \sum_{|\alpha| \le t} L_{ij,\alpha} \frac{y^{\alpha}}{\alpha!}.$$

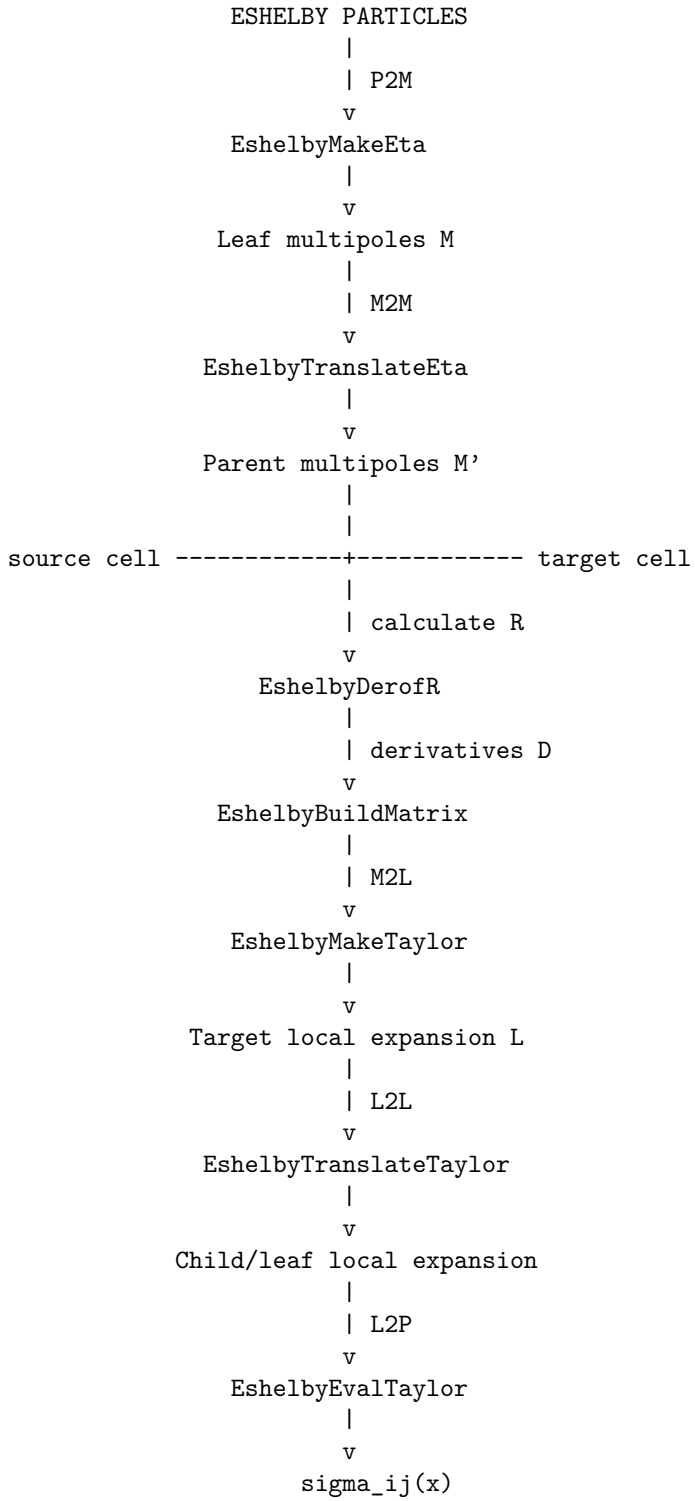
The output is the symmetric stress matrix

$$\sigma = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{xz} \\ \sigma_{xy} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{xz} & \sigma_{yz} & \sigma_{zz} \end{bmatrix}.$$

This is effectively the **L2P — local-to-point** stage.

11. The complete Eshelby FMM in one diagram

The entire far-field process can be summarized as:



12. Where does the dislocation force enter?

The FMM path produces the **stress field**

$$\sigma(x).$$

Once stress is known, the dislocation force can be obtained through the Peach–Koehler relation

$$\boxed{g = (\sigma b) \times t,}$$

where

- b is the Burgers vector,
- t is the dislocation line direction.

So conceptually:

$$\boxed{\text{Eshelby particles} \rightarrow \text{FMM stress} \rightarrow \text{Peach–Koehler force on dislocation.}}$$

13. Near field versus far field

This distinction is important in ParaDiS. For **far-away inclusions**, use the FMM:

$$\boxed{\text{MakeEta} \rightarrow \text{TranslateEta} \rightarrow \text{MakeTaylor} \rightarrow \text{TranslateTaylor} \rightarrow \text{EvalTaylor.}}$$

For a nearby inclusion and a dislocation segment, the code can instead use

$$\boxed{\texttt{EshelbyForce.cc}}$$

which performs an analytic line integration and returns endpoint forces directly. It bypasses the FMM expansion hierarchy. So the conceptual division is

$$\boxed{\begin{cases} \text{near field} & \rightarrow \text{direct interaction,} \\ \text{far field} & \rightarrow \text{FMM.} \end{cases}}$$

14. Why all these transformations help

Suppose there are N inclusions and N target locations. A naive approach evaluates approximately

$$N^2$$

source-target interactions. FMM instead says:

> If 100 particles are very far from me, I do not need to evaluate all 100 individually. I can replace them with one multipole representation of their cell.

And similarly:

> If many targets lie near one another, I can construct one local Taylor expansion for their cell and reuse it.

That is why the sequence has two compact representations:

$$\boxed{M_\alpha}$$

for the **source side**, and

$$\boxed{L_{ij,\alpha}}$$

for the **target side**.

15. A useful way to remember every routine

FMM stage	ParaDiS routine	Meaning
P2M	`EshelbyMakeEta`	Particle → multipole
M2M	`EshelbyTranslateEta`	Child multipole → parent multipole
Kernel	`EshelbyDerofR`	Derivatives of $1/r$
Packing	`EshelbyBuildMatrix`	Arrange derivative/Taylor tensors for contractions
M2L	`EshelbyMakeTaylor`	Source multipole → target Taylor
L2L	`EshelbyTranslateTaylor`	Parent Taylor → child Taylor
L2P	`EshelbyEvalTaylor`	Taylor → stress at actual point
Near-field direct	`EshelbyForce`	Inclusion → nodal segment forces

The shortest summary is therefore:

$$\boxed{\text{Particle} \to M \to M' \to L \to L' \to \sigma(x)}$$

or, in FMM language,

$$\boxed{P2M \to M2M \to M2L \to L2L \to L2P.}$$

One implementation caution from the reviewed source is worth keeping in mind: the documentation identifies sign-direction discrepancies in the current `TranslateEta` and `TranslateTaylor` center shifts, so the sequence above describes the intended mathematical FMM architecture, while the literal translation routines have odd-shift sign issues documented separately.